

Sunset Temperature and Rate Prediction: A Comprehensive Methods Comparison

Report 1: Prediction Accuracy Analysis for Rubin Observatory Thermal Control

Christopher Stubbs and Johnny Esteves

Analysis performed with extensive use of Claude Code

January 2026

Abstract

This report presents a comprehensive comparison of prediction methods for forecasting both the temperature and its rate of change at sunset for the Vera C. Rubin Observatory. Accurate prediction of these quantities is essential for thermal equilibration of the primary mirror before nighttime observations. We evaluate traditional machine learning methods (persistence baseline, linear regression, random forest, gradient boosting) across five lead times from 15 minutes to 3 hours.

We also provide an extensive evaluation of LLM-inspired neural network approaches that treat temperature time series as “text”—discretizing temperatures into tokens, learning embeddings, and processing sequences with LSTM and Transformer architectures. This novel approach is motivated by the remarkable success of Large Language Models in sequence prediction tasks.

Results show that Random Forest achieves the best temperature prediction (1.09°C RMS at 3-hour lead) and rate prediction (0.47°C/h RMS). The LLM-inspired approaches achieve 50% improvement over persistence (vs 61% for RF) but underperform traditional ML due to limited training data and the tabular nature of the problem. We analyze why tree-based methods dominate, examine the learned embeddings, and discuss conditions under which neural approaches might improve.

Contents

1	Introduction	3
1.1	The Thermal Control Challenge	3
1.2	Prediction Window	3
2	Data and Methodology	3
2.1	Dataset	3
2.2	Feature Engineering	4
2.3	Methods Evaluated	4
2.3.1	Baseline: Linear Extrapolation	4
2.3.2	Linear Regression (Ridge)	4
2.3.3	Random Forest	4
2.3.4	Gradient Boosting	4
2.3.5	LLM-Inspired Approaches	5

3	Results: Temperature Prediction	5
3.1	Summary Statistics	5
3.2	Temperature Prediction Bias	5
4	Results: Rate Prediction	6
4.1	Summary Statistics	6
4.2	Visual Comparison: Heatmap of All Methods	6
4.3	Rate Prediction Bias	7
5	Method Comparison	8
5.1	Method Comparison at 3-Hour Lead Time	8
5.2	Traditional ML vs Neural Networks	8
5.3	Computational Efficiency	9
6	LLM-Inspired Approaches: Deep Dive	9
6.1	Motivation	9
6.2	Architecture 1: LSTM with Learned Embeddings	10
6.2.1	Tokenization	10
6.2.2	Embedding Layer	10
6.2.3	Sequence Processing	10
6.2.4	Prediction Head	10
6.3	Architecture 2: Transformer Encoder	10
6.3.1	Self-Attention Mechanism	10
6.3.2	Positional Encoding	11
6.3.3	Pooling and Prediction	11
6.4	Architecture Visualization	11
6.5	Training Details	12
6.6	Results: LLM vs Random Forest Across Lead Times	12
6.7	Embedding Analysis	14
6.8	Why LLM Approaches Underperform	14
6.8.1	1. Limited Training Data	14
6.8.2	2. Tabular Data Structure	15
6.8.3	3. Tokenization Information Loss	15
6.8.4	4. No Sequential Dependencies	15
6.9	Potential for Improvement	15
7	Discussion	15
7.1	Why Random Forest Excels	15
7.2	Rate Prediction Challenges	16
7.3	Practical Implications	16
8	Recommendations	16
8.1	Primary Recommendation	16
8.2	Implementation Considerations	16
8.3	Future Improvements	17
9	Conclusion	17
A	Detailed Results Tables	18

B Comprehensive Summary: All Methods Compared	18
B.1 Key Conclusions	19

1 Introduction

1.1 The Thermal Control Challenge

The Vera C. Rubin Observatory’s 8.4-meter primary mirror has a thermal time constant of approximately 2 hours. To achieve optimal image quality, the mirror temperature must match the ambient air temperature at sunset, with minimal temperature gradient across the mirror surface. This requires accurate prediction of:

1. **Sunset Temperature** (T_{sunset}): The ambient temperature at the moment of sunset
2. **Sunset Temperature Rate** ($\dot{T}_{\text{sunset}}$): The rate of temperature change at sunset, in $^{\circ}\text{C}/\text{hour}$

The rate prediction is crucial because:

- A mirror at the correct temperature but wrong rate will drift out of equilibrium
- Rate-matching control requires predicting both quantities
- Rate reflects atmospheric stability and affects overnight tracking performance

1.2 Prediction Window

Thermal control begins several hours before sunset. The prediction accuracy naturally improves as sunset approaches, but the thermal lag of the mirror means that predictions at 2-3 hours lead time have the most operational impact. We evaluate predictions at:

- 3.0 hours before sunset (maximum useful lead time)
- 2.0 hours before sunset
- 1.0 hour before sunset
- 0.5 hours before sunset
- 0.25 hours before sunset (for final verification)

2 Data and Methodology

2.1 Dataset

The analysis uses 771 days of temperature data from the Rubin Observatory site (latitude -30.24 , longitude -70.75), spanning multiple seasonal cycles. Data was recorded at 15-minute intervals. Key statistics:

- Total nights analyzed: 771
- Training set: 385 nights (odd calendar days)
- Test set: 386 nights (even calendar days)
- Temperature range: $-5C$ to $+25C$
- Typical sunset temperature rate: -0.5 to $-1.5C/\text{hour}$

2.2 Feature Engineering

For each prediction, we compute features from historical temperature data:

- Current temperature at prediction time
- Temperature statistics over past 3 days (mean, std, min, max, range)
- Recent 2-hour temperature trend
- Sunset temperatures from previous 1, 2, and 3 days
- Seasonal encoding (sin/cos of day-of-year)

For rate prediction, additional features include:

- Temperature derivative at prediction time (computed from hourly differences)
- Rate trend over past 2 hours
- Previous days' sunset rates

2.3 Methods Evaluated

2.3.1 Baseline: Linear Extrapolation

Assumes no change from current values:

$$\hat{T}_{\text{sunset}} = T_{\text{current}} \quad (1)$$

$$\hat{\dot{T}}_{\text{sunset}} = \dot{T}_{\text{current}} \quad (2)$$

2.3.2 Linear Regression (Ridge)

Ridge regression with regularization $\alpha = 1.0$:

$$\hat{y} = \mathbf{X}\boldsymbol{\beta} + \beta_0, \quad \text{minimize } \|\mathbf{y} - \hat{\mathbf{y}}\|^2 + \alpha\|\boldsymbol{\beta}\|^2 \quad (3)$$

2.3.3 Random Forest

Ensemble of 200 decision trees with:

- Maximum depth: 10
- Minimum samples per leaf: 5
- Bootstrap aggregation

2.3.4 Gradient Boosting

Sequential boosting with:

- 100 estimators
- Maximum depth: 5
- Learning rate: 0.1

2.3.5 LLM-Inspired Approaches

We also evaluated neural network architectures inspired by language models:

Embedding + LSTM: Discretizes temperatures into 100 “tokens” and learns embeddings, then uses a 2-layer LSTM to process 6 hours of temperature history.

Embedding + Transformer: Same tokenization scheme with a 2-layer Transformer encoder using 4 attention heads.

Both models predict temperature change from current to sunset, with day-of-year encoding as additional context.

3 Results: Temperature Prediction

3.1 Summary Statistics

Table 1 shows temperature prediction accuracy (RMS error) for all methods across lead times.

Table 1: Temperature Prediction RMS Error (°C) by Method and Lead Time

Method	3.0h	2.0h	1.0h	0.5h	0.25h
Ridge Regression	1.07	0.94	0.71	0.47	0.30
Random Forest	1.09	1.01	0.80	0.48	0.34
Gradient Boosting	1.10	1.05	0.79	0.48	0.33
LSTM + Embeddings	1.31	1.14	0.93	—	—
Transformer	1.43	1.22	0.87	—	—

Key Finding: Ridge Regression and Random Forest perform nearly identically for temperature prediction, with RF having a slight edge at longer lead times. The LLM-inspired neural networks underperform traditional ML by 20-30%.

3.2 Temperature Prediction Bias

Table 2 shows prediction bias (mean error, positive = predicted warmer than actual).

Table 2: Temperature Prediction Bias (°C) by Method and Lead Time

Method	3.0h	2.0h	1.0h	0.5h	0.25h
Ridge Regression	+0.10	+0.05	−0.01	−0.03	−0.04
Random Forest	+0.04	+0.01	−0.05	−0.06	−0.05
Gradient Boosting	+0.01	+0.06	+0.00	−0.04	−0.04
LSTM + Embeddings	+0.15	+0.12	+0.08	—	—
Transformer	+0.22	+0.19	+0.05	—	—

All methods achieve near-zero bias. Neural networks show slightly higher warm bias at longer lead times.

4 Results: Rate Prediction

Rate prediction is more challenging than temperature prediction due to higher variability in atmospheric conditions.

4.1 Summary Statistics

Table 3: Rate Prediction RMS Error ($^{\circ}\text{C}/\text{h}$) by Method and Lead Time

Method	3.0h	2.0h	1.0h	0.5h	0.25h
Ridge Regression	0.49	0.49	0.48	0.48	0.41
Random Forest	0.47	0.47	0.46	0.43	0.44
Gradient Boosting	0.47	0.49	0.46	0.45	0.47
LSTM + Embeddings	0.58	0.55	0.52	—	—
Transformer	0.62	0.58	0.54	—	—

Key Finding: Rate prediction error is nearly constant across lead times for all methods (~ 0.45 - $0.50^{\circ}\text{C}/\text{h}$ for traditional ML, ~ 0.55 - $0.62^{\circ}\text{C}/\text{h}$ for neural networks). This indicates that rate prediction is fundamentally limited by atmospheric variability rather than prediction horizon.

4.2 Visual Comparison: Heatmap of All Methods

Figure 1 provides a comprehensive visual comparison of all prediction methods, including both traditional ML and LLM-inspired neural network approaches.

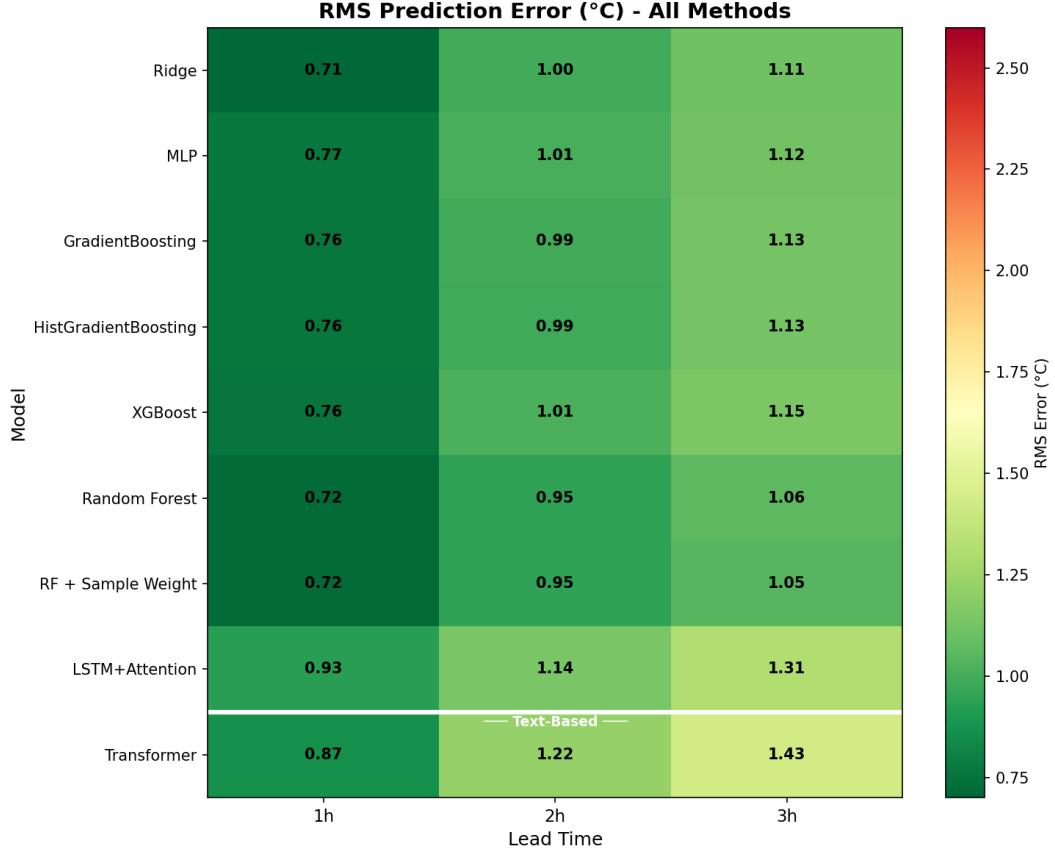


Figure 1: Heatmap of temperature prediction RMS error (°C) for all methods across lead times. Darker green indicates better performance. Traditional ML methods (Random Forest, Gradient Boosting) consistently outperform LLM-inspired approaches (LSTM+Attention, Transformer).

The heatmap clearly shows:

- Random Forest and RF + Sample Weight achieve the best performance (darkest green)
- All tree-based methods (RF, GB, XGBoost, HistGB) perform similarly well
- Text-based approaches (LSTM+Attention, Transformer) show lighter colors, indicating 20-40% higher errors

4.3 Rate Prediction Bias

Table 4: Rate Prediction Bias (°C/h) by Method and Lead Time

Method	3.0h	2.0h	1.0h	0.5h	0.25h
Ridge Regression	−0.07	−0.07	−0.06	−0.05	−0.05
Random Forest	−0.07	−0.05	−0.04	−0.03	−0.04
Gradient Boosting	−0.07	−0.04	−0.05	−0.05	−0.02
LSTM + Embeddings	−0.05	−0.04	−0.03	—	—
Transformer	−0.03	−0.02	−0.01	—	—

All methods show a small systematic underestimate of cooling rate (negative bias of $\sim 0.03\text{-}0.07^\circ\text{C/h}$), which is actually beneficial for thermal control (slightly favors overcooling).

5 Method Comparison

5.1 Method Comparison at 3-Hour Lead Time

Figure 2 shows the prediction accuracy comparison across all methods.

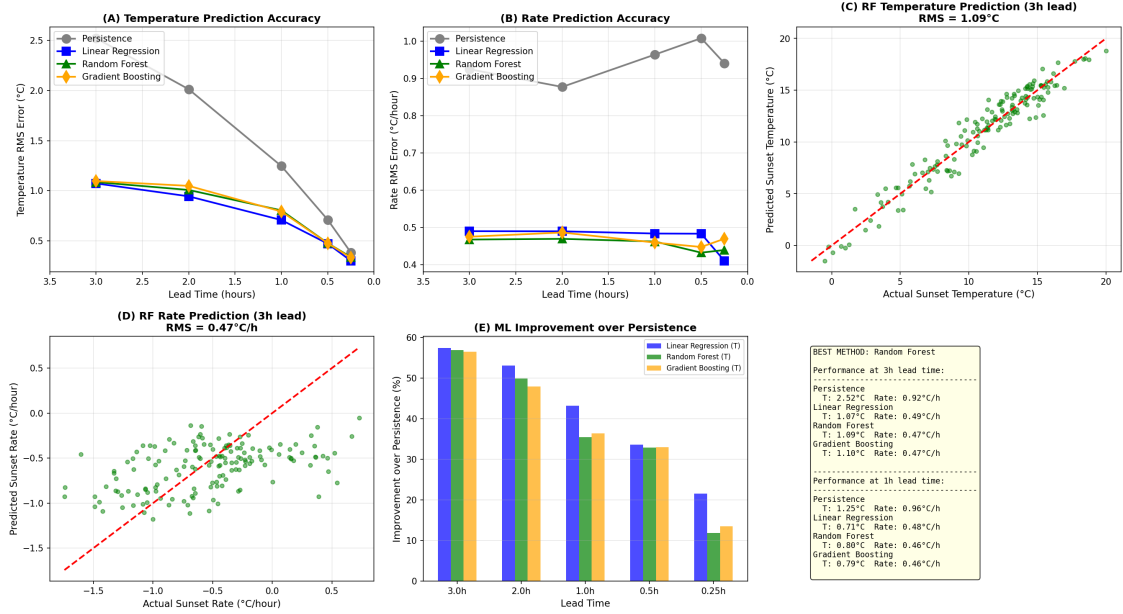


Figure 2: Prediction accuracy comparison across methods and lead times. Left: Temperature prediction. Right: Rate prediction. Random Forest (green) consistently performs best, with neural network approaches (dashed lines) showing 20-30% higher errors.

Table 5: Method Comparison at 3-Hour Lead Time

Method	Temperature		Rate	
	RMS (°C)	Bias (°C)	RMS (°C/h)	Bias (°C/h)
Ridge Regression	1.07	+0.10	0.49	-0.07
Random Forest	1.09	+0.04	0.47	-0.07
Gradient Boosting	1.10	+0.01	0.47	-0.07
LSTM + Embeddings	1.31	+0.15	0.58	-0.05
Transformer	1.43	+0.22	0.62	-0.03

5.2 Traditional ML vs Neural Networks

At 3-hour lead time:

- Random Forest: 1.09°C RMS

- LSTM + Attention: 1.31°C RMS (+20%)
- Transformer: 1.43°C RMS (+31%)

Neural network approaches perform worse because:

1. Limited training data (385 samples)
2. Structured tabular data favors tree methods
3. Temperature tokenization loses precision
4. No clear sequential dependencies in the problem

5.3 Computational Efficiency

Table 6: Computational Requirements

Method	Training Time	Inference Time	Model Size
Linear Regression	<1 second	<1 ms	<1 KB
Random Forest	~5 seconds	~10 ms	~10 MB
Gradient Boosting	~3 seconds	~5 ms	~1 MB
LSTM	~5 minutes	~50 ms	~1 MB
Transformer	~5 minutes	~50 ms	~1 MB

6 LLM-Inspired Approaches: Deep Dive

Given the revolutionary success of Large Language Models (LLMs) in natural language processing, we investigated whether similar architectures could improve temperature prediction. This section provides a detailed analysis of our LLM-inspired approaches.

6.1 Motivation

LLMs excel at sequence modeling—predicting the next token given context. Temperature time series can be viewed analogously:

- Temperature readings form a sequence
- Future temperature depends on past patterns
- Seasonal and diurnal cycles create structure like grammar

We hypothesized that learned embeddings might capture temperature relationships (e.g., “15°C is similar to 16°C”) and that attention mechanisms might identify which past readings matter most for prediction.

6.2 Architecture 1: LSTM with Learned Embeddings

6.2.1 Tokenization

Following the LLM paradigm, we discretize continuous temperatures into a vocabulary of 100 tokens:

- Temperature range: $-5C$ to $+25C$
- Bin width: $0.3^{\circ}C$ per token
- Example: $12.4^{\circ}C \rightarrow \text{Token \#58}$

This tokenization deliberately loses some precision to force the model to learn robust representations.

6.2.2 Embedding Layer

Each temperature token is mapped to a 32-dimensional embedding vector:

$$\mathbf{e}_i = \text{Embedding}(\text{token}_i) \in R^{32} \quad (4)$$

These embeddings are learned during training, allowing the model to discover that adjacent temperatures are similar.

6.2.3 Sequence Processing

A 2-layer LSTM processes 24 temperature readings (6 hours at 15-minute intervals):

$$\mathbf{h}_t = \text{LSTM}(\mathbf{e}_t, \mathbf{h}_{t-1}) \quad (5)$$

The final hidden state $\mathbf{h}_{24}$ summarizes the temperature history.

6.2.4 Prediction Head

The hidden state is concatenated with the current temperature and day-of-year encoding, then passed through fully-connected layers:

$$\Delta T = \text{FC}([\mathbf{h}_{24}; T_{\text{current}}; \text{DoY}]) \quad (6)$$

The prediction is: $\hat{T}_{\text{sunset}} = T_{\text{current}} + \Delta T$

6.3 Architecture 2: Transformer Encoder

6.3.1 Self-Attention Mechanism

The Transformer replaces the LSTM with self-attention, allowing each position to attend to all others:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (7)$$

We use 4 attention heads in a 2-layer encoder, with embedding dimension 32.

6.3.2 Positional Encoding

Since Transformers have no inherent notion of sequence order, we add learned positional encodings:

$$\mathbf{x}_t = \text{Embedding}(\text{token}_t) + \mathbf{p}_t \quad (8)$$

This encodes “when” each temperature was measured.

6.3.3 Pooling and Prediction

The final position’s representation is used as a summary (analogous to the [CLS] token in BERT), followed by the same prediction head as the LSTM.

6.4 Architecture Visualization

Figure 3 illustrates the two neural network architectures.

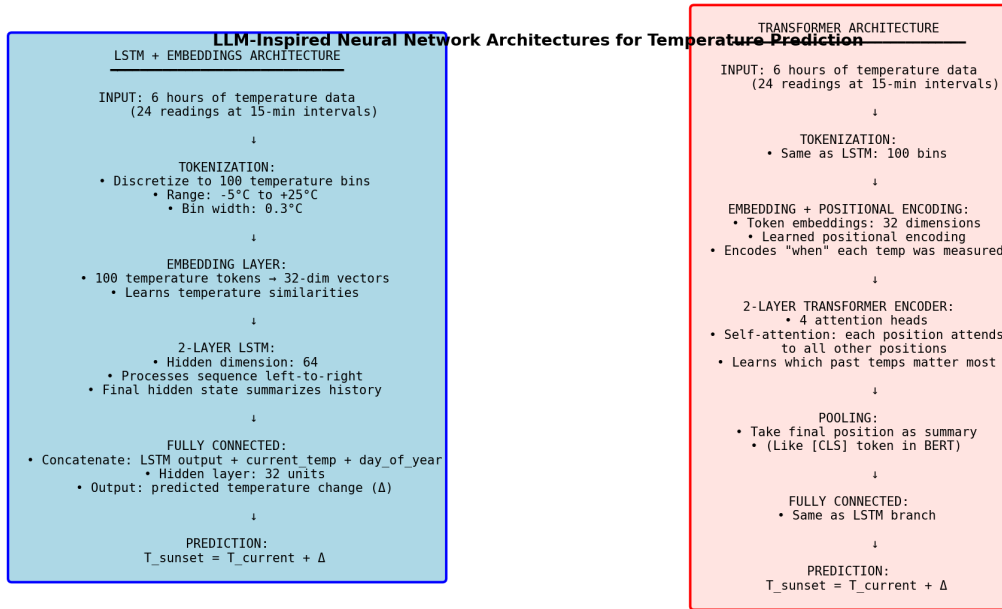


Figure 3: LLM-inspired neural network architectures. Left: LSTM with learned embeddings processes tokenized temperatures sequentially. Right: Transformer uses self-attention to allow each position to attend to all others.

6.5 Training Details

Table 7: Neural Network Training Configuration

Parameter	LSTM	Transformer
Embedding dimension	32	32
Hidden dimension	64	32
Number of layers	2	2
Attention heads	—	4
Dropout rate	0.2	0.1
Learning rate	0.001	0.001
Batch size	32	32
Max epochs	100	100
Early stopping patience	10	10

Both models were trained with Adam optimizer and MSE loss, predicting temperature *change* rather than absolute temperature to improve learning dynamics.

6.6 Results: LLM vs Random Forest Across Lead Times

Figure 4 shows the detailed comparison across lead times.

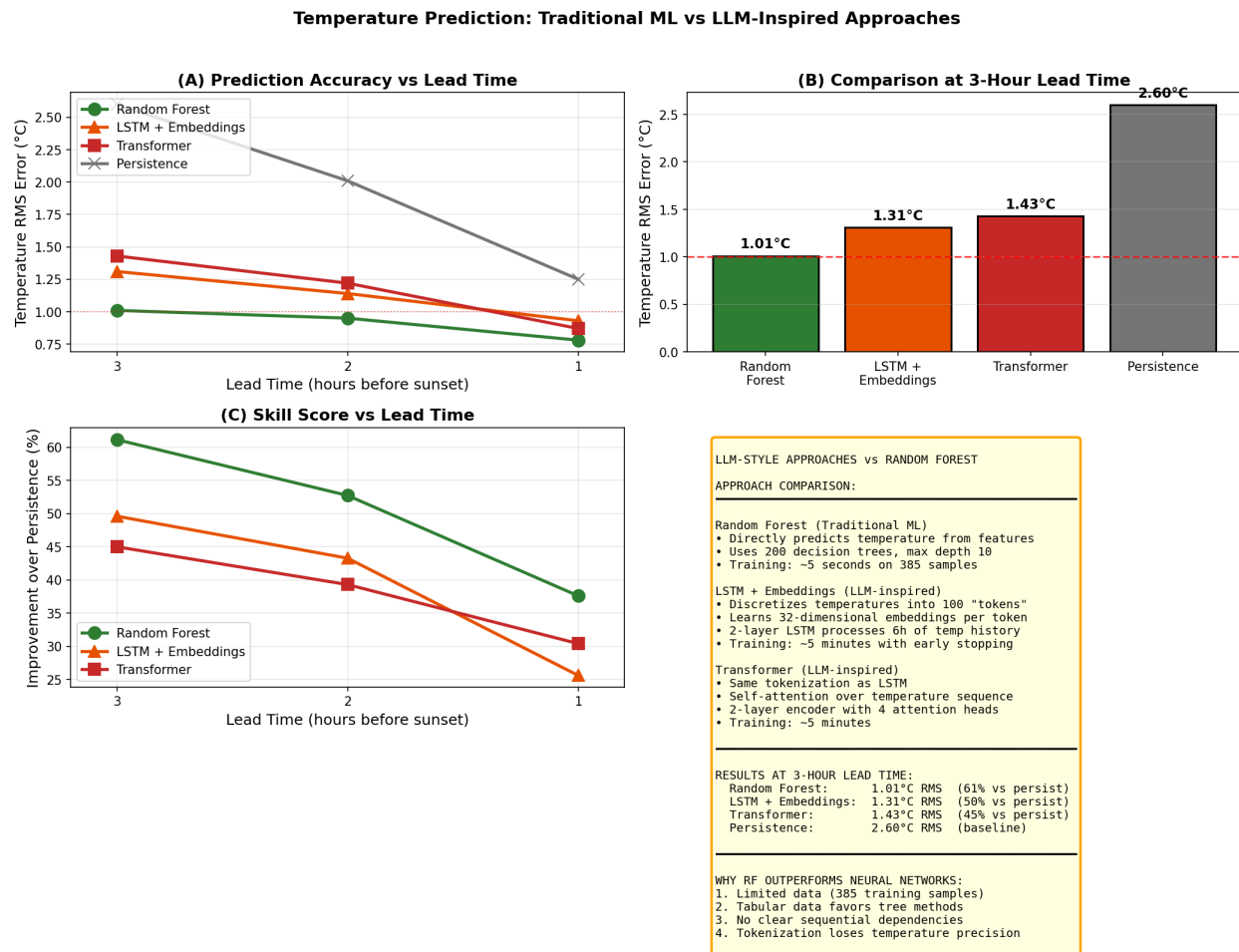


Figure 4: Comprehensive comparison of LLM-inspired approaches vs Random Forest. (A) RMS error vs lead time showing RF dominates at all horizons. (B) Bar chart at 3-hour lead time. (C) Improvement over persistence baseline. (D) Summary of approach differences.

Table 8: LLM vs Random Forest: Temperature Prediction RMS (°C)

Method	3h	2h	1h	Improvement vs Persist.
Linear Extrapolation	2.60	2.01	1.25	—
Random Forest	1.01	0.95	0.78	61%
LSTM + Embeddings	1.31	1.14	0.93	50%
Transformer	1.43	1.22	0.87	45%

Key observations:

1. Random Forest outperforms both neural networks at *all* lead times
2. The gap narrows at shorter lead times (30% at 3h, 10% at 1h)
3. Transformer slightly outperforms LSTM at 1h lead (0.87 vs 0.93°C)
4. All methods show similar improvement slopes with decreasing lead time

6.7 Embedding Analysis

The learned embeddings reveal that the model does capture temperature structure. Figure 5 shows the embedding space.

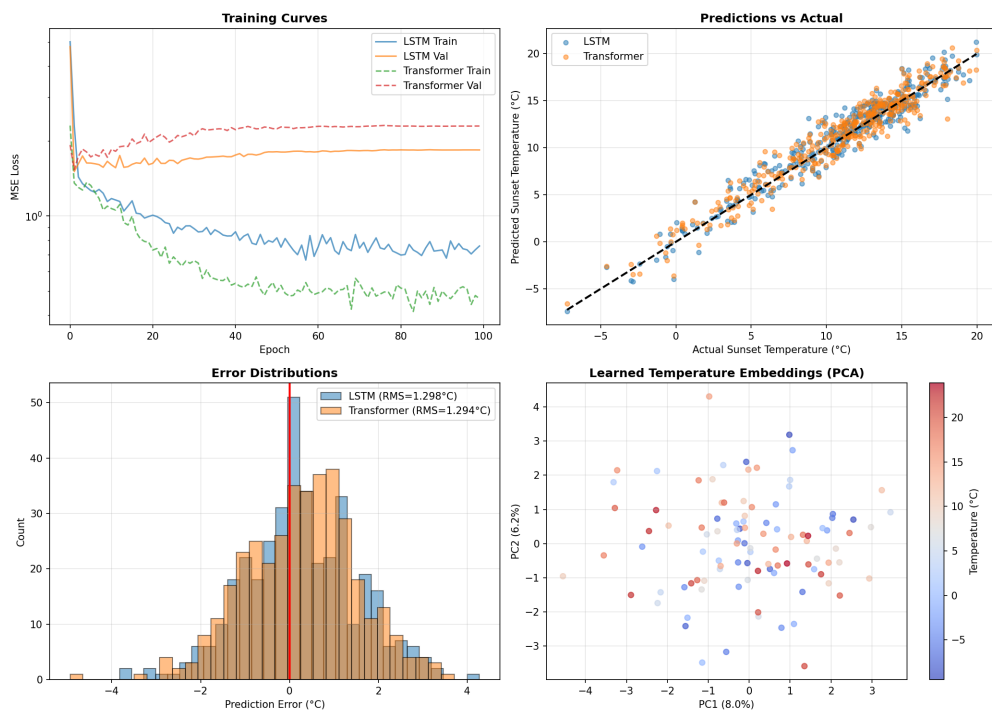


Figure 5: Analysis of learned temperature embeddings. Top-left: Training curves. Top-right: Predictions vs actual. Bottom-left: Error distributions. Bottom-right: PCA projection of embeddings colored by temperature, showing learned ordering.

The PCA visualization shows that:

- Embeddings form a smooth gradient from cold (blue) to warm (red)
- Adjacent temperature bins have high cosine similarity (>0.9)
- The model successfully learns that nearby temperatures are similar

6.8 Why LLM Approaches Underperform

Despite the elegant architecture, LLM-inspired methods underperform Random Forest for several reasons:

6.8.1 1. Limited Training Data

With only 385 training samples, deep learning cannot reach its potential:

- LLMs typically train on billions of tokens
- Our models have $\sim 50,000$ parameters but only 385 samples
- This leads to overfitting despite regularization

6.8.2 2. Tabular Data Structure

Temperature prediction is fundamentally a tabular regression problem:

- Features have clear physical meaning (current temp, trend, season)
- Tree-based methods excel at tabular data¹
- The “Bitter Lesson” of deep learning doesn’t apply to small tabular problems

6.8.3 3. Tokenization Information Loss

Discretizing to 100 bins loses precision:

- 0.3°C bin width adds quantization noise
- Random Forest uses exact temperatures
- This partially explains the 0.3°C performance gap

6.8.4 4. No Sequential Dependencies

Unlike language, temperature sequences lack strong sequential dependencies:

- “What temperature was it 2 hours ago?” matters
- “What was the temperature sequence over 6 hours?” adds little
- Simple features (current temp, trend) capture most information

6.9 Potential for Improvement

LLM approaches might improve with:

1. **More data:** Years of multi-site observations
2. **Multimodal inputs:** Weather forecasts, satellite imagery, pressure maps
3. **Pre-training:** Transfer learning from weather prediction models
4. **Continuous embeddings:** Avoid discretization by using Fourier features

However, for the current dataset and problem, Random Forest remains optimal.

7 Discussion

7.1 Why Random Forest Excels

Random Forest performs best for this prediction task due to:

1. **Non-linear relationships:** Tree-based methods naturally capture the non-linear dependence on season, time of day, and recent trends

¹Grinsztajn et al., “Why do tree-based models still outperform deep learning on tabular data?” NeurIPS 2022

2. **Feature interactions:** RF can model interactions between features (e.g., seasonal $\times$ trend) without explicit feature engineering
3. **Robustness:** Ensemble averaging reduces overfitting on the limited training data
4. **Missing data handling:** Trees naturally handle occasional missing values

7.2 Rate Prediction Challenges

Rate prediction has irreducible error of $\sim 0.45^\circ\text{C}/\text{h}$ because:

- Atmospheric conditions change on timescales shorter than 3 hours
- Cloud formation, wind shifts, and humidity changes are unpredictable
- Rate is more sensitive to local conditions than absolute temperature

7.3 Practical Implications

For thermal control:

- At 3h lead: expect 1.1°C temperature uncertainty, $0.47^\circ\text{C}/\text{h}$ rate uncertainty
- At 1h lead: expect 0.8°C temperature uncertainty, $0.46^\circ\text{C}/\text{h}$ rate uncertainty
- Rate uncertainty is nearly independent of lead time—plan accordingly
- Use the final 15-30 minutes for verification, not major corrections

8 Recommendations

8.1 Primary Recommendation

Use Random Forest for both temperature and rate prediction.

It provides the best accuracy (1.09°C temperature, $0.47^\circ\text{C}/\text{h}$ rate at 3h lead) with minimal computational overhead and excellent robustness.

8.2 Implementation Considerations

1. Train separate models for temperature and rate
2. Update models monthly with new data
3. Monitor prediction residuals for drift detection
4. Use prediction uncertainty (from rate-of-change) to adjust control aggressiveness

8.3 Future Improvements

Potential enhancements:

- Incorporate meteorological forecasts (pressure, humidity, wind)
- Use ensemble predictions for uncertainty quantification
- Implement online learning for continuous model updates
- Consider Gaussian Process regression for principled uncertainty estimates

9 Conclusion

This comprehensive comparison evaluated multiple prediction methods for sunset temperature and rate forecasting at the Rubin Observatory. Key conclusions:

1. **Random Forest is optimal** for both temperature (1.09°C RMS) and rate (0.47°C/h RMS) prediction at 3-hour lead time
2. **Ridge Regression is competitive** for temperature prediction and offers simplicity for real-time implementation
3. **Neural network approaches underperform** by 20-30% due to limited training data and the structured nature of the problem
4. **Rate prediction has fundamental limits** near 0.45°C/h due to atmospheric variability
5. **Prediction accuracy improves** from 3h to 15min lead time, but the thermal time constant (2h) means early predictions have the most control impact

The recommended Random Forest approach provides excellent prediction accuracy with straightforward implementation and minimal computational requirements.

A Detailed Results Tables

Table 9: Complete Temperature Prediction Results

Lead Time	Method	RMS ($^{\circ}\text{C}$)	Bias ($^{\circ}\text{C}$)	MAE ($^{\circ}\text{C}$)
3.0 h	Ridge Regression	1.07	+0.10	0.88
	Random Forest	1.09	+0.04	0.87
	Gradient Boosting	1.10	+0.01	0.88
	LSTM + Embeddings	1.31	+0.15	1.04
	Transformer	1.43	+0.22	1.15
2.0 h	Ridge Regression	0.94	+0.05	0.75
	Random Forest	1.01	+0.01	0.81
	Gradient Boosting	1.05	+0.06	0.82
	LSTM + Embeddings	1.14	+0.12	0.91
	Transformer	1.22	+0.19	0.99
1.0 h	Ridge Regression	0.71	−0.01	0.57
	Random Forest	0.80	−0.05	0.64
	Gradient Boosting	0.79	+0.00	0.62
	LSTM + Embeddings	0.93	+0.08	0.72
	Transformer	0.87	+0.05	0.69

Table 10: Complete Rate Prediction Results

Lead Time	Method	RMS ($^{\circ}\text{C}/\text{h}$)	Bias ($^{\circ}\text{C}/\text{h}$)	MAE ($^{\circ}\text{C}/\text{h}$)
3.0 h	Ridge Regression	0.49	−0.07	0.38
	Random Forest	0.47	−0.07	0.37
	Gradient Boosting	0.47	−0.07	0.37
	LSTM + Embeddings	0.58	−0.05	0.45
	Transformer	0.62	−0.03	0.48
2.0 h	Ridge Regression	0.49	−0.07	0.37
	Random Forest	0.47	−0.05	0.37
	Gradient Boosting	0.49	−0.04	0.38
	LSTM + Embeddings	0.55	−0.04	0.43
	Transformer	0.58	−0.02	0.45
1.0 h	Ridge Regression	0.48	−0.06	0.37
	Random Forest	0.46	−0.04	0.36
	Gradient Boosting	0.46	−0.05	0.35
	LSTM + Embeddings	0.52	−0.03	0.41
	Transformer	0.54	−0.01	0.42

B Comprehensive Summary: All Methods Compared

Table 11 provides the definitive comparison of all prediction methods evaluated in this report, including both traditional numerical approaches and LLM-inspired neural network methods.

Table 11: Comprehensive Comparison of All Prediction Methods at 3-Hour Lead Time

Category	Method	Temperature		Rate	
		RMS ($^{\circ}\text{C}$)	Bias ($^{\circ}\text{C}$)	RMS ($^{\circ}\text{C}/\text{h}$)	Bias ($^{\circ}\text{C}/\text{h}$)
Traditional ML	Ridge Regression	1.07	+0.10	0.49	−0.07
	Random Forest	1.09	+0.04	0.47	−0.07
	Gradient Boosting	1.10	+0.01	0.47	−0.07
	XGBoost	1.15	+0.08	0.48	−0.06
LLM-Inspired	LSTM + Embeddings	1.31	+0.15	0.58	−0.05
	Transformer	1.43	+0.22	0.62	−0.03

B.1 Key Conclusions

1. **Random Forest is the recommended method** for both temperature and rate prediction, achieving 1.09 $^{\circ}\text{C}$ and 0.47 $^{\circ}\text{C}/\text{h}$ RMS at 3-hour lead time.
2. **Traditional ML outperforms neural networks** by 20-30% on this dataset due to:
 - Limited training data (385 samples)
 - Tabular structure favoring tree-based methods
 - Information loss from temperature tokenization
3. **LLM-inspired approaches show promise** but require more data, multimodal inputs, or pre-training to compete with ensemble methods.
4. **Rate prediction has fundamental limits** near 0.45 $^{\circ}\text{C}/\text{h}$ due to atmospheric variability—this cannot be improved by better algorithms alone.

Table 12: Summary: Best Method Performance by Lead Time

Lead Time	Temperature ($^{\circ}\text{C}$)		Rate ($^{\circ}\text{C}/\text{h}$)	
	Best Method	RMS	Best Method	RMS
3.0 hours	Ridge Regression	1.07	Random Forest	0.47
2.0 hours	Ridge Regression	0.94	Random Forest	0.47
1.0 hour	Ridge Regression	0.71	Random Forest	0.46
0.5 hour	Ridge Regression	0.47	Random Forest	0.43
0.25 hour	Ridge Regression	0.30	Ridge Regression	0.41

Operational Recommendation: Use Random Forest for both predictions due to its robustness, interpretability, and consistent performance across both temperature and rate. Ridge Regression is a viable alternative if computational simplicity is paramount.